

Worksheet — 2.3 Algorithms — ANSWER SHEET

Separate answer sheet / mark scheme — keep for marking.

Answer key

Activity 1 — Big O match

Algorithm	Best	Average	Worst
Linear search	$O(1)$	$O(n)$	$O(n)$
Binary search	$O(1)$	$O(\log n)$	$O(\log n)$
Bubble sort (with swap flag)	$O(n)$	$O(n^2)$	$O(n^2)$
Insertion sort	$O(n)$	$O(n^2)$	$O(n^2)$
Merge sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
Quick sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$

Note: bubble sort best case is $O(n)$ **only with a swap flag** on already-sorted data. Quick sort worst case is $O(n^2)$ with a poor pivot (e.g. already-sorted data, first/last element pivot).

Activity 2 — Sort trace

BUBBLE SORT of [5, 1, 4, 2, 8] (compare adjacent pairs, swap if left > right; largest bubbles to the end each pass):

Pass	List state after this pass	Swaps this pass
Start	5 1 4 2 8	—
1	1 4 2 5 8	$5 \leftrightarrow 1$, $5 \leftrightarrow 4$, $5 \leftrightarrow 2$ = 3 swaps
2	1 2 4 5 8	$4 \leftrightarrow 2$ = 1 swap
3	1 2 4 5 8	0 swaps → with swap flag, algorithm stops here

Final sorted list: 1 2 4 5 8

Pass-by-pass detail of pass 1 (each adjacent comparison): - (5,1) → swap → 1 5 4 2 8 - (5,4) → swap → 1 4 5 2 8 - (5,2) → swap → 1 4 2 5 8 - (5,8) → no swap → 1 4 2 5 8

INSERTION SORT alternative (acceptable if student chose insertion) — list after each pass (element inserted into sorted front):

Pass	List state after this pass
Start	5 1 4 2 8
1 (insert 1)	1 5 4 2 8
2 (insert 4)	1 4 5 2 8
3 (insert 2)	1 2 4 5 8
4 (insert 8)	1 2 4 5 8

Final sorted list: 1 2 4 5 8 (same result).

Activity 3 — Binary search for 27

List: indices 0–8, values 3, 7, 11, 15, 19, 23, 27, 31, 35.

Step	low	high	mid (index)	value at mid	< = > target?
1	0	8	4	19	19 < 27 → search right, low = mid+1 = 5
2	5	8	6	27	27 = 27 → FOUND

Found at index: **6**. Number of comparisons: **2**.

(Working: step 1 mid = (0+8) DIV 2 = 4; step 2 mid = (5+8) DIV 2 = 6.)

Activity 4 — Justifications

(a) Binary search is $O(\log n)$ because **each comparison halves the size of the list still to be searched** — it discards the half that cannot contain the target. The number of times n can be halved before reaching 1 is $\log_2(n)$, so the number of comparisons grows logarithmically with n . (Doubling the list size adds only one extra comparison.)

(b) Bubble sort is $O(n^2)$ because it uses **two nested loops**: an outer loop of up to n passes, and for each pass an inner loop making up to n adjacent comparisons. That gives roughly $n \times n = n^2$ comparisons, so time grows with the square of the input size.

(c) Binary search requires the list to be **sorted**; linear search works on unsorted data.

Activity 5 — Tree traversals

Tree: root 40; left subtree 20 (children 10, 30); right subtree 60 (children 50, 70).

- **Pre-order** (Node, Left, Right): 40, 20, 10, 30, 60, 50, 70
- **In-order** (Left, Node, Right): 10, 20, 30, 40, 50, 60, 70
- **Post-order** (Left, Right, Node): 10, 30, 20, 50, 70, 60, 40

Ascending order is produced by the **in-order** traversal of a BST.

Activity 6 — Dijkstra A to F

Edges (undirected): A-B 4, A-C 2, B-C 1, B-D 5, C-D 8, C-E 10, D-E 2, D-F 6, E-F 3.

Settle the unvisited node with the smallest tentative distance each round; relax its neighbours.

Settled (order)	A	B	C	D	E	F
Initial	0	∞	∞	∞	∞	∞
Settle A (0)	0	4	2	∞	∞	∞
Settle C (2)	0	3	2	10	12	∞
Settle B (3)	0	3	2	8	12	∞
Settle D (8)	0	3	2	8	10	14
Settle E (10)	0	3	2	8	10	13
Settle F (13)	0	3	2	8	10	13

Relaxation notes: - From A: B = 4, C = 2. - Settle C (2): B = $\min(4, 2+1) = 3$; D = $2+8 = 10$; E = $2+10 = 12$.
- Settle B (3): D = $\min(10, 3+5) = 8$; (C already settled). - Settle D (8): E = $\min(12, 8+2) = 10$; F = $8+6 = 14$.
- Settle E (10): F = $\min(14, 10+3) = 13$. - Settle F (13): done.

Shortest path A → F: A → C → B → D → E → F

Total distance: 13.

(Check: A → C 2, C → B 1, B → D 5, D → E 2, E → F 3 = $2+1+5+2+3 = 13$. Compare alternative
A → C → B → D → F = $2+1+5+6 = 14$, longer. So 13 is optimal.)

Challenge

(a) A heuristic is **admissible** if it **never overestimates** the true shortest-distance from a node to the goal F. To check, compare each $h(n)$ to the actual shortest distance to F (computed by Dijkstra from each node): - True shortest distances to F — F:0, E:3, D:5 (D → E → F = $2+3$), B:10 (B → D → E → F = $5+2+3$), C:11 (C → B → D → E → F = $1+5+2+3$), A:13. - Heuristic given: A=9, B=7, C=8, D=4, E=3, F=0. Each is $\leq$ the true distance ($9 \leq 13$, $7 \leq 10$, $8 \leq 11$, $4 \leq 5$, $3 \leq 3$, $0 \leq 0$), so the heuristic is **admissible** and A* would find the optimal path.

(b) A explores fewer nodes than Dijkstra when a good (informative) heuristic directs the search toward the goal instead of expanding outward in all directions — useful when you only need the path to one goal node. A behaves **exactly like Dijkstra** when the heuristic is **$h(n) = 0$** for every node (no information), because then $f(n) = g(n)$.

Activity 7 — Merge sort diagram

(a) Missing boxes:

- (i) [51 24 9 16]
- (ii) [43 8]
- (iii) [8 43]
- (iv) [9 16 24 51]
- (v) [8 9 12 16 24 35 43 51]

Full merge stage: singles → [12 35] [8 43] [24 51] [9 16] → [8 12 35 43] [9 16 24 51] → [8 9 12 16 24 35 43 51].

(Check final merge, comparing front elements: $8 < 9 \rightarrow 8$; $12 > 9 \rightarrow 9$; $12 < 16 \rightarrow 12$; $35 > 16 \rightarrow 16$; $35 > 24 \rightarrow 24$; $35 < 51 \rightarrow 35$; $43 < 51 \rightarrow 43$; 51 remains. ✓)

(b) 3 levels of dividing ($8 \rightarrow 4 \rightarrow 2 \rightarrow 1$). This is $\log_2 8 = 3$ — the number of times 8 can be halved to reach 1.

(c) Each merge level processes **every element once**, so roughly n comparisons/copies per level ($\approx O(n)$ work). With $\log_2 n$ levels each costing $O(n)$, the total is $n \times \log n \rightarrow O(n \log n)$.

(d) Merge sort needs **$O(n)$ additional memory** for the temporary lists used while merging (it is not in-place), and it does the full $n \log n$ work even on already-sorted data.

Activity 8 — A* vs Dijkstra

Edges: S-A 1, S-B 4, S-D 2, A-B 2, A-C 5, B-C 2, C-G 3. Heuristic: $h(S)=7$, $h(A)=6$, $h(B)=4$, $h(C)=3$, $h(D)=9$, $h(G)=0$.

(a) Dijkstra from S:

Settled (order)	S	A	B	C	D	G
Initial	0	∞	∞	∞	∞	∞
Settle S (0)	0	1	4	∞	2	∞
Settle A (1)	0	1	3	6	2	∞
Settle D (2)	0	1	3	6	2	∞
Settle B (3)	0	1	3	5	2	∞
Settle C (5)	0	1	3	5	2	8
Settle G (8)	0	1	3	5	2	8

Relaxations: from S: $A=1$, $B=4$, $D=2$. Settle A(1): $B=\min(4, 1+2)=3$, $C=1+5=6$. Settle D(2): only neighbour S already settled — **no updates**. Settle B(3): $C=\min(6, 3+2)=5$. Settle C(5): $G=5+3=8$. Settle G(8).

Settle order: **S → A → D → B → C → G** (all 6 nodes; Dijkstra settles the dead-end D third).

(b) A* from S:

Node expanded	$g(n)$	$h(n)$	$f(n)$
S	0	7	7
A	1	6	7
B	3	4	7
C	5	3	8
G	8	0	8

Working: expand S → open list $A(f=1+6=7)$, $B(f=4+4=8)$, $D(f=2+9=11)$. Expand A(7) → B improves to $g=3$, $f=7$; C added $g=6$, $f=9$. Expand B(7) → C improves to $g=5$, $f=8$. Expand C(8) → G added $g=8$, $f=8$. Expand G(8) = goal, stop.

Shortest path S → G: **S → A → B → C → G**, distance **8** ($1+2+2+3 = 8$; alternatives $S \rightarrow B \rightarrow C \rightarrow G = 4+2+3 = 9$ and $S \rightarrow A \rightarrow C \rightarrow G = 1+5+3 = 9$ are longer ✓).

(c) D. Its f value is $f(D) = g(D) + h(D) = 2 + 9 = 11$, which is **higher than the f of every node on the route to the goal** (all ≤ 8). Because A *always* expands the lowest- f node and reaches G at $f = 8$, D is

never the best choice — the heuristic signals that *D* is heading away from the goal, so *A* never wastes time expanding it. (The heuristic is admissible: each $h(n) \leq$ the true distance to *G* — $S \leq 8$, $A \leq 7$, $B \leq 5$, $C \leq 3$, $D \leq 10$, $G \leq 0$ — so *A**'s answer is still optimal.)

(d) Satnav (one start, one destination, straight-line distances available): **A*** — the admissible heuristic focuses the search towards the destination, so it typically expands **fewer nodes** than Dijkstra for a single goal. Network-monitoring tool (shortest routes to **every** node): **Dijkstra** — it naturally computes shortest distances to **all** nodes in one run, and no single-goal heuristic exists when every node is a target.

Section B — model answers

Q1 (2 marks). - Worst case: **$O(n^2)$** . **[1]** - Occurs when the pivot repeatedly splits the data as unevenly as possible (sublists of size 0 and $n-1$) — e.g. choosing the first/last element as pivot on data that is **already sorted (or reverse sorted)**. **[1]**

Q2 (3 marks). - The complexity is **$O(n^2)$** . **[1]** - There are **two nested loops**: the outer loop runs n times, and for **each** outer iteration the inner loop runs n times. **[1]** - So the `print` statement executes $n \times n = n^2$ times; the running time grows with the **square** of the input size (constant factors are ignored in Big O). **[1]**

Q3 (4 marks).

(i) One mark per correct row **[3]**:

low	high	mid	arr[mid]	found
0	7	3	22	false
0	2	1	9	false
2	2	2	15	false

Working: iteration 1 — $mid = (0+7) \text{ DIV } 2 = 3$; $arr[3] = 22 > 10 \rightarrow high = 2$. Iteration 2 — $mid = (0+2) \text{ DIV } 2 = 1$; $arr[1] = 9 < 10 \rightarrow low = 2$. Iteration 3 — $mid = (2+2) \text{ DIV } 2 = 2$; $arr[2] = 15 > 10 \rightarrow high = 1$. Now $low (2) > high (1)$, so the loop ends with `found = false`.

(ii) The value printed is **-1** (the target 10 is not in the array). **[1]**

Q4 (5 marks). Model answer:

```

function searchIndex(arr:array, n, target)
    low = 0
    high = n - 1
    while low <= high
        mid = (low + high) DIV 2
        if arr[mid] == target then
            return mid
        elseif arr[mid] < target then
            low = mid + 1
        else
            high = mid - 1
        endif
    endwhile
    return -1
endfunction

```

Mark points (one each): - Function declaration with the three parameters; **low/high** pointers **initialised** to 0 and $n - 1$. **[1]** - Loop that continues **while low <= high**. **[1]** - Midpoint calculated with **integer division**: $\text{mid} = (\text{low} + \text{high}) \text{ DIV } 2$. **[1]** - Correct **narrowing** of the search range: $\text{low} = \text{mid} + 1$ when $\text{arr}[\text{mid}] < \text{target}$, $\text{high} = \text{mid} - 1$ when it is greater. **[1]** - **Returns mid** when the target is found and **returns -1** after the loop when it is not. **[1]**

Q5 (2 marks). - (i) Breadth-first: a **queue** (FIFO). **[1]** - (ii) Depth-first: a **stack** (LIFO) — either an explicit stack or the implicit **call stack** via recursion. **[1]**

Q6 (3 marks). - (i) **A, B, C, D, E**. **[2]** (2 marks fully correct; 1 mark for one error/omission.) Working: visit A, enqueue B, C; visit B, enqueue D; visit C, enqueue E; visit D (E already queued); visit E. - (ii) BFS explores the graph in **levels** — all nodes 1 edge away, then all nodes 2 edges away, and so on — so the **first** time it reaches E it must have used the fewest possible edges; DFS plunges down one branch first (e.g. $A \rightarrow B \rightarrow D \rightarrow E$) and may reach a node by a **longer** route. **[1]**

Q7 (2 marks). - $g(n)$ is the **actual cost of the path from the start node to n** found so far. **[1]** - $h(n)$ is the **heuristic estimate of the cost from n to the goal** (which must not overestimate — admissible — for A to *guarantee the optimal path*). **[1]***

Q8* (9 marks) — levels of response.

- **Level 3 (7–9 marks):** Thorough knowledge and understanding of searching and sorting algorithms and their complexities. Choices are **justified with correct Big O for both time and space** and clearly **applied to all three parts of the scenario** (fast searching, nightly server sort, memory-limited device). Well-developed, logically structured line of reasoning with a reasoned conclusion.
- **Level 2 (4–6 marks):** Reasonable knowledge and understanding. Sensible algorithm choices with some justification and mostly correct complexities; partial application to the scenario (e.g. ignores the memory constraint). Some structure to the reasoning.
- **Level 1 (1–3 marks):** Basic knowledge only. Names algorithms with little or no justification, incorrect or missing complexities, generic statements not applied to the scenario.
- **0 marks:** No response worthy of credit.

Model top-band answer:

Searching. The 50 million records are stored **in order of track ID**, which is the precondition for **binary search**. Binary search is **$O(\log n)$** : each comparison halves the remaining search space, so a lookup needs at most about $\log_2(50,000,000) \approx 26$ **comparisons**, compared with up to 50 million for a **linear search ($O(n)$)**. At thousands of searches per second, linear search would be completely infeasible, so binary search should be used everywhere the data is sorted — including the car system's list of downloaded tracks.

Nightly sort on the server. For 50 million records, an **$O(n^2)$** algorithm such as bubble or insertion sort is unusable ($\approx 2.5 \times 10^{15}$ comparisons), so an **$O(n \log n)$** algorithm is needed. **Merge sort** is the best choice for the server: its worst case is **guaranteed $O(n \log n)$** (unlike quicksort, which degrades to **$O(n^2)$** on unluckily ordered data — a risk when the input is already mostly sorted), so the nightly job will reliably finish inside its time window. Merge sort's drawback is **$O(n)$ auxiliary space** for the temporary merge lists, but the scenario states the server has a large amount of memory, so this cost is acceptable. Better still, the developers can exploit the structure of the data: the existing 50 million records are **already sorted** and only a few thousand new tracks arrive. Sorting just the small batch ($n \log n$ on a few thousand items) and then performing a **single $O(n)$ merge pass** into the main file is far cheaper than re-sorting everything — exactly the merge step of merge sort.

Embedded car system. Here memory is the constraint, so merge sort's $O(n)$ extra space is a poor fit. With only a few hundred tracks, an **in-place** algorithm with **$O(1)$ auxiliary space**, such as **insertion sort**, is appropriate: although it is $O(n^2)$ in the average/worst case, n is tiny so the time is negligible, and insertion sort runs in close to **$O(n)$** when new tracks are inserted into an already-sorted list — the common case here. (An in-place quicksort would also be acceptable.)

Conclusion: binary search ($O(\log n)$) for all lookups on the sorted data; merge sort — ideally sort-the-batch-then-merge — for the nightly server job because its guaranteed $O(n \log n)$ time matters more than its $O(n)$ space there; and in-place insertion sort on the embedded device, where $O(1)$ space matters more than asymptotic speed on a few hundred items.